In [7]:
```python
#Create a function that takes a list of integers and returns a dictionary with the count of
#even numbers, odd numbers, and their respective sums.
l1 = [1,2,3,4,5,6,7]
def dict_even_odd(l):
    d = dict()
    c_e = 0
    c_o = 0
    s_e=0
    s_o = 0
    for i in l:
        if i%2==0:
            c_e+=1
            s_e+=i
        if i%2!=0:
            c_o+=1
            s_o+=i
    d['even'] = c_e
    d['odd'] = c_o
    d['Sum of even'] = s_e
    d['Sum of odd'] = s_o

    return d
x = dict_even_odd(l1)
x


```

Out[7]:
```
{'even': 3, 'odd': 4, 'Sum of even': 12, 'Sum of odd': 16}
```

In [63]:
```
#Write a recursive function to reverse a string without using built-in methods like [::-1],
#and handle edge cases like empty strings or single characters.
def rev_str(s,i):
    if len(s) <=1:
        return s


    if i==len(s)+1:
        return ''
    return s[len(s)-i]+rev_str(s,i+1)


s = 'abcd'
i = 1

result = rev_str(s,i)
result
```

Out[63]: 'dcba'

In [45]:
```
#Design a function that accepts a mixed list of integers and strings, filters out
#non-numeric values, and returns the product of all remaining numbers.

def prod_f(l):
    prod = 1
    l1 = [x for x in l if isinstance(x,int)]
    for i in l1:
            prod*=i
    return l1,prod
l1 = [2,5,7,8,"Hello","Bye",1]
l1,r = prod_f(l1)
print(l1,r)

```

```
[2, 5, 7, 8, 1] 560
```

In [65]:
```
#Develop a function to simulate a bank account: it takes an initial balance and a list of
#transactions (positive for deposits, negative for withdrawals), returning the final balance
#or an error if it goes negative.
def bank_op(balance,transactions):
    if balance<0:
        return "Insufficient Balance"
    f_balance=balance
    for i in transactions:
        f_balance+=i
    return f_balance

r = bank_op(1000,[10,-40,190,-21])
r
```

Out[65]: 1139

In [ ]:
```
#Create a function that takes a string and extracts all consecutive digit sequences (e.g.,
#"abc123xyz45"→ ["123","45"]), returning them as a list.
str1 = 'abc123xyz45'
def extract_num(s):
    l1 = []
    n = ''
    for i in range(len(s)):
        if s[i].isdigit():
            n+=s[i]
```

In [120]:

```python
#Write a function using **kwargs to define a custom calculator supporting addition,
#subtraction, multiplication, and division, with error handling for invalid operations.
def calc(**kwargs):
    values = [x for x in kwargs.values() if isinstance(x,int)]
    sm = 0
    df = values[0]
    prod = 1
    dv = values[0]
    if 'op' in kwargs.keys():
        if kwargs['op'] == 'add':
            for i in values:
                sm+=i
            return sm
        if kwargs['op'] == 'subtract':
            for i in values[1:]:
                df -= i
            return df
        if kwargs['op'] == 'multiply':
            for i in values:
                prod*=i
            return prod
        if kwargs['op'] == 'divide':
            for i in values[1:]:
                if i==0:
                    raise ZeroDivisionError
                dv/=i
            return dv
        else:
            raise 'Invalid Operation'

print('Sum:',calc(x=1,y=2,z=4,op='add'))
print('Difference:',calc(x=1,y=2,z=4,op='subtract'))
print('Sum:',calc(x=1,y=2,z=4,op='multiply'))
print('Sum:',calc(x=1,y=9,z=4,op='divide'))
```

```
Sum: 7
Difference: -5
Sum: 8
Sum: 0.027777777777777776
```

In [100]:
```python
#Build a recursive function to compute the sum of digits in a nested list (e.g., [1, [2, 3], [4,
#[5]]] → 15).
def sum_list(l):
    sm = 0
    for i in l:
        if isinstance(i,list):
            sm+=sum_list(i)
        else:
            sm+=i
    return sm
print(sum_list([1, [2, 3], [4,[5]]]))

```

```
15
```

In [109]:
```python
#Design a function that takes a list of numbers and returns the second-largest value,
#handling cases with duplicates or fewer than two elements.
def second(l):
    if len(l)<2:
        return l
    unique = list(set(l))
    mx = max(unique)
    unique.remove(mx)
    mx = max(unique)
    return mx
print(second([1,2,3,45,6,6]))

```

```
6
```

In [119]:
```python
#Create a real-life function to calculate the total travel time across multiple legs of a trip,
#where each leg is a tuple of (distance, speed), returning time in hours.
def travel_time(l):
    total = 0
    for i in l:
        t = i[1]/i[0]
        total+=t
    return total
print(travel_time([(100,80),(20,50)]))
```

```
3.3
```

In [121]:
```python
#Write a function that generates all prime factors of a given number using recursion (e.g.,
#12 → [2, 2, 3]).
def prime(n,i=2):
    if n==1:
        return []
    if n%i==0:
        return[i] + prime(n//i,i)
    return prime(n,i+1)
print(prime(12))
```

```
[2, 2, 3]
```

In [141]:
```
#Develop a function to parse a log file string and count occurrences of specific keywords,
#returning a dictionary (e.g.,"error: timeout, warning: retry"→ {"error": 1,"warning": 1})
def parser(log):
    l = log.split(', ')
    keywords = []
    r = {}
    for i in l:
        index = i.find(':')
        keywords.append(i[:index])
    for j in l:
        c = 0
        index = j.find(':')
        for k in keywords:
            if j[:index] in k:
                c+=1
                r[k]=c
    return r
parser("error: timeout, warning: retry")

['error: timeout','warning: retry']


```

Out[141]: {'error': 1, 'warning': 1}

In [148]:
```
#Create a function that takes a list of integers and finds the longest increasing
#subsequence (e.g., [10, 9, 2, 5, 3, 7, 101, 18] → [2, 3, 7, 101]).
def lic(l):
    lc = []
    for i in range(len(l)):
        if l[i] not in lc:
            lc.append(l[i])
        for j in range(i+1):
            if l[i]>l[j]:
                lc.append(l[j])
    print(lc)
```

In [152]:
```
#Write a recursive function to flatten a deeply nested list into a single-level list (e.g., [1, [2,
#[3, 4], 5]] → [1, 2, 3, 4, 5]).
def flat(l):
    f = []
    for i in l:
        if isinstance(i,list):
            f.extend(flat(i))
        else:
            f.append(i)
    return f
print(flat([1, [2,[3, 4], 5]]))
```

```
[1, 2, 3, 4, 5]
```

In [157]:
```
#Design a function to simulate a vending machine: it takes a dictionary of items with
#prices and stock, processes a purchase, and updates stock or returns an error.
def vending(items):
    choice = 'orange juice'
    if choice=='orange juice':
        price = items['orange juice']['price']
        items['orange juice']['stock'] = items['orange juice']['stock'] -1
        return price,items
    elif choice=='apple juice':
        price = items['apple juice']['price']
        items['apple juice']['stock'] = items['apple juice']['stock'] -1
        return price,items
    elif choice=='monster':
        price = items['monster']['price']
        items['monster']['stock'] = items['monster']['stock'] -1
        return price,items
items = {'orange juice':{'price':20,'stock':12},'apple juice':{'price':20,'stock':14},'monster':{'price':
amount, new_stock = (vending(items))
print("Amount to be paid:",amount)
print("Updated stock:",new_stock)


```

```
Amount to be paid: 20
Updated stock: {'orange juice': {'price': 20, 'stock': 11}, 'apple juice': {'price': 20, 'stock': 14}, 'mon
ster': {'price': 60, 'stock': 8}}
```

In [200]:
```
#Build a function that takes a string of parentheses and checks if it’s balanced, supporting
#multiple types like (), [], {} (e.g.,"{[()}]"→ False).
def paran(s):
    while '()' in s or "{}" in s or '[]' in s:
        s = s.replace('()','').replace('[]','').replace('{}','')
    return s==''


paran('{[()}]')
```

Out[200]: False

In [231]:
```python
#Write a function using *args to process a variable number of student grade inputs
#(numeric or letter grades like "A"), converting them to points and averaging them.
def average(*args):
    total = 0

    for grade in args:
        if isinstance(grade,int):
            if grade>90 and grade<=100:
                total+=5.0
            elif grade>80 and grade<=90:
                total+=4.0
            elif grade>70 and grade<=80:
                total+=3.0
            elif grade>60 and grade<=70:
                total+=2.0
            elif grade>50 and grade<=60:
                total+=1.0
            else:
                return "FAIL"
        if isinstance(grade,str):
            if grade=='A':
                total+=5.0
            elif grade=='B':
                total+=4.0
            elif grade=='C':
                total+=3.0
            elif grade=='D':
                total+=2.0
            elif grade=='E':
                total+=1.0
            else:
                return "FAIL"
    average = total/len(args)
    return average
average('A',70,'B',60)
```

Out[231]: 3.0

In [237]:

```
#Create a real-life function to calculate compound interest over multiple periods, taking
#principal, rate, time, and compounding frequency as inputs.
def compound_interest(P,r,t,n):
    r = r /100
    amount = P*(1+r/n)**(n*t)
    interest = amount - P
    return round(amount,2), round(interest,2)

a,i = compound_interest(1000,5,3,4)
print("Amount received:",a)
print("Interest received:",i)

```

```
Amount received: 1160.75
Interest received: 160.75
```

In [252]:

```
#Develop a function to find all unique triplets in a list of integers that sum to zero
#(e.g., [-1,0, 1, 2,-1,-4] → [[-1, 0, 1], [-1, 2,-1]]).
def find_triplets(l1):
    l1.sort()
    re = []
    for i in range(len(l1)-2):
        if i>0 and l1[i] == l1[i-1] :
            continue
        l = i+1
        r = len(l1)-1
        while l<r:
            total = l1[i] + l1[l] +l1[r]
            if total == 0:
                re.append([l1[i],l1[l],l1[r]])
                while l < r and l1[l] == l1[l + 1]:
                    l += 1
                while l < r and r>0 and l1[r] == l1[r - 1]:
                    r -= 1

                l += 1
                r -= 1
            elif total < 0:
                l += 1
            else:
                r -= 1


    return re
find_triplets([-1,0, 1, 2,-1,-4])
```

Out[252]:

```
[[-1, -1, 2], [-1, 0, 1]]
```

In [255]:
```
#Write a recursive function to compute the number of ways to climb a staircase with n
#steps, where you can take 1 or 2 steps at a time.
def climb(n):
    if n==0:
        return 1
    if n==1:
        return 1
    if n==2:
        return 2
    return climb(n-1)+climb(n-2)
climb(10)
```

Out[255]: 89

In [ ]:
```

```